

Indian Institute of Technology Bombay

EE739: Processor Design

Project 1

IITB-RISC Superscalar Processor

2-Way Fetch · Out-of-Order Execution

Team ID: 01

Team Members:

Yashvardhan Khandegar	23B1227
Sudhindra Sahoo	23B1312
Vinay Hariharan	23B1310
Himansu Sahu	23B3972

Date: May 12, 2026

Contents

1 Introduction **5**

1.1 Project Objective 5

1.2 ISA Summary 5

1.3 Design Assumptions 5

2 Superscalar Processor Architecture **6**

2.1 Design Assumptions 6

2.2 Front-End: Instruction Fetch and Prediction 7

2.2.1 Program Counter (PC) 7

2.2.2 Instruction Memory (IMEM) 7

2.2.3 Branch Predictor (BP Unit) 7

2.2.4 MF Unit (Multiple-Access Microcode Sequencer) 8

2.2.5 Instruction Fetch Buffers (IF1 and IF2) 8

2.3 Decode and Register Renaming 9

2.3.1 Decode Units 9

2.3.2 Register Renaming Unit (RR Unit) 9

2.3.3 Dispatch Stage 10

2.4 Instruction Scheduling and Issue 10

2.4.1 Scheduler 10

2.4.2 ALU Reservation Station 11

2.4.3 Branch Reservation Station 11

2.4.4 Load/Store Queue (LSQ) 11

2.5 Execution Units 12

2.5.1 ALU Pipeline 12

2.5.2 Branch / Next-Address Block 12

2.5.3 Load/Store Pipeline 13
2.6 Reorder Buffer (ROB) 13
2.7 Commit and Retirement 14
2.7.1 Register Result File (RRF) 14
2.7.2 Architectural Register File (ARF) 15
2.7.3 Condition Code Register (CCR) 15
2.8 Summary of Key Microarchitectural Features 15

3 Detailed Unit Descriptions **16**
3.1 Superscalar Processor Architecture 16
3.2 Fetch Unit 16
3.2.1 Functionality and Port Groups 16
3.2.2 Next-PC Priority Logic (Flowchart) 17
3.2.3 Combinational vs. Sequential Logic 19
3.2.4 Why the BP Uses the PC, Not the Instruction 19
3.3 BP_Unit — Branch Target Buffer 20
3.3.1 BTB Structure and Indexing 20
3.3.2 2-Bit Saturating Counter FSM 20
3.3.3 Two-Process Design 20
3.4 Decode Unit 21
3.4.1 Functionality 21
3.5 RR_Unit — Register Rename 21
3.5.1 Overview 21
3.5.2 Rename Logic Flowchart 23
3.5.3 0-Cycle RAW Forwarding 23
3.5.4 Input/Output Port Summary 24
3.6 Dispatch Router 24
3.6.1 Routing Logic 24
3.7 ALU_RS — Reservation Station 26
3.7.1 Three-Phase Per-Cycle Operation 26
3.8 ALU_Exec — Execution Unit 27
3.8.1 Operation and Hold/Freeze Mechanism 27

3.8.2 Supported Operations . . . 27
3.9 ROB — Re-Order Buffer . . . 28
3.9.1 Entry Fields . . . 28
3.9.2 Commit Logic . . . 28
3.10 MF_Sequencer . . . 28

4 LOAD & STORE in our Out-of-Order Processor 29
4.1 Overview and Architectural Challenges . . . 29
4.2 The Front-End: CISC-to-RISC Translation . . . 29
4.3 The Load/Store Queue (LSQ) . . . 30
4.4 Memory Disambiguation and Execution . . . 30
4.5 The Commit Phase and State Recovery . . . 30
4.6 VHDL Implementation Considerations . . . 31

5 Branch & Jump in our Out-of-Order Processor 32
5.1 Overview and Architectural Challenges . . . 32
5.2 The Speculative Leap: Fetch & Predict . . . 32
5.3 Decode & Rename: The JAL Exception . . . 33
5.4 The Waiting Room: Branch_RS . . . 33
5.5 Execution: The Moment of Truth . . . 33
5.6 The Reorder Buffer: The Judge & Executioner . . . 34
5.7 Architectural Advantages . . . 34

6 Simulation Results 36
6.1 Arithmetic and Logical Instructions . . . 36
6.1.1 Instruction Memory and Simulation . . . 36
6.1.2 Output of Fetch Unit and MF Sequencer . . . 37
6.1.3 Decode Channel 1 (After 1 Cycle) . . . 37
6.1.4 Decode Channel 2 (After 1 Cycle) . . . 38
6.1.5 Register Renaming (After 2 Cycles) . . . 38
6.1.6 Final Architectural Register File (ARF) Values . . . 39
6.2 Fetch Unit Verification Results . . . 40
6.2.1 Simulation Console Output . . . 40

6.2.2 Simulation Waveform 41
6.3 Branch Misprediction and Pipeline Flush Verification 41
6.3.1 Simulation Waveform 41
6.4 Branch Prediction (BP) Unit Verification 43
6.4.1 Simulation Console Output 43
6.5 Branch Not-Taken Verification 44
6.5.1 Simulation Waveform 44
6.6 LMF Instruction Verification 45

7 Key Design Decisions **46**
7.1 Word-Addressed IMEM with Synchronous Read 46
7.2 Dual CDB 46
7.3 Combinational BP Read, Synchronous BP Write 46
7.4 TAG Stack for Branch Aliasing 46
7.5 MF Sequencer for Multi-Register Instructions 47
7.6 Zero-Cycle Intra-Bundle RAW Forwarding in RR_Unit 47

8 Work Distribution **48**

9 Conclusion **49**

1. Introduction

1.1. Project Objective

This report documents the design and VHDL implementation of a 2-way fetch, out-of-order execution superscalar processor based on the IITB-RISC instruction set architecture.

The IITB-RISC is a 16-bit, 8-register architecture. This superscalar implementation fetches two instructions per cycle and can commit up to two instructions per cycle, targeting maximum instruction-level parallelism.

1.2. ISA Summary

Parameter	Value
Word Width	16 bits
Registers	8 general-purpose (R0–R7)
Condition Codes	Carry (C), Zero (Z)
Instruction Formats	R-type, I-type, J-type
Total Instructions	28
Addressing	Byte-addressable

1.3. Design Assumptions

- Normal PC increment: PC $\leftarrow$ PC + 4.
- ARF: 8 registers. RRF (physical registers): 16 entries.
- ROB depth: configurable (default 16 entries).
- Dual CDB: two result buses active simultaneously.
- BLE Opcode: 1011

2. Superscalar Processor Architecture

This chapter describes the microarchitecture of the 2-way superscalar, out-of-order processor designed for the IITB-RISC instruction set. The processor is built around Tomasulo's algorithm and supports dual instruction issue, speculative execution with branch prediction, and precise exception handling via a Reorder Buffer (ROB). The overall pipeline follows the sequence:

Fetch $\rightarrow$ **Decode** $\rightarrow$ **Rename** $\rightarrow$ **Dispatch** $\rightarrow$ **Schedule** $\rightarrow$ **Execute** $\rightarrow$ **ROB** $\rightarrow$ **Commit**

A high-level block diagram of the complete microarchitecture is shown in Figure **??**.

2.1. Design Assumptions

The following assumptions govern the implementation:

- The processor is a **fixed-point** machine; no floating-point support is required.
- Memory is **byte-addressable**. A 16-bit (2-byte) word is fetched with two consecutive byte reads: $\{M[\text{addr}],\ M[\text{addr}+1]\}$.
- The **normal PC** advances by 4 bytes per cycle ($\text{PC}_{\text{next}} = \text{PC} + 4$), fetching two instructions simultaneously.
- There are **three functional unit pipelines** in the execute stage: an ALU pipeline (single-cycle), a branch/next-address pipeline, and a load/store pipeline (two cycles).
- The **BLE** instruction is assigned opcode `1011`.
- The **RRF** contains **16 physical registers** (tags `p0`–`p15`), while the ARF retains the 8 architectural registers R0–R7.

2.2. Front-End: Instruction Fetch and Prediction

2.2.1. Program Counter (PC)

The Program Counter generates the instruction fetch address each cycle. Because two 16-bit instructions are fetched per cycle, the sequential increment is PC + 4 (byte-addressed). Three sources may write the PC, resolved in priority order:

1. **Misprediction recovery** — the correct PC produced by the ROB at commit, accompanied by a pipeline flush (`WC = 1`).
2. **Branch prediction target** — the target supplied by the BP Unit when a branch is detected in the fetch bundle.
3. **Sequential increment** — PC + 4 when no branch is detected.

2.2.2. Instruction Memory (IMEM)

The Instruction Memory is a synchronous, dual-read memory that delivers two consecutive 16-bit instructions in a single cycle, forming a 32-bit fetch bundle $\{I_1, I_2\}$ at addresses PC and PC + 2 respectively. The memory accepts the PC on one clock edge and presents valid instructions on the following edge.

2.2.3. Branch Predictor (BP Unit)

The Branch Predictor enables speculative, uninterrupted instruction flow by predicting control transfers before the branch is executed. It is implemented as a **Branch Target Buffer (BTB)** with the following structure:

Table 2.1. BTB entry fields

Field	Width	Description
TAG	6 bits	PC bits used to index and identify the branch (last 6 bits of the branch instruction address).
HB	2 bits	2-bit saturating history counter: `00` = Strongly Not Taken (SNT), `01` = Weakly Not Taken (WNT), `10` = Weakly Taken (WT), `11` = Strongly Taken (ST).
TAKEN PC	16 bits	Predicted target address when taken.
NOT TAKEN PC	16 bits	Fall-through address (PC + 4).
BUSY	1 bit	Entry validity flag.

Output PC logic. When a branch is detected (`bc = 1`):

- If `HB[1] = 1` (Taken state): `out_PC` ← `TAKEN_PC`.
- Otherwise: `out_PC` ← `PC` + 4.

History table update logic. At commit, when the actual branch outcome (`br`) and the write-enable signal (`wc`) are known, the history bits are updated according to the standard 2-bit saturating counter FSM (SNT ↔ WNT ↔ WT ↔ ST).

Correctness constraint. If the *first* instruction in a fetch bundle is predicted taken, the *second* instruction (I_2) is invalidated before decode, since it lies on the wrong control-flow path.

2.2.4. MF Unit (Multiple-Access Microcode Sequencer)

The MF Unit handles the IITB-RISC multi-register memory instructions: `LM`, `LMF`, `SM`, and `SMF`. Each such instruction implicitly accesses up to 8 registers in consecutive memory locations, which cannot be executed atomically in a single pipeline pass. The unit decomposes them into a stream of individual `LW`/`SW` micro-operations.

The MF Unit operates as a two-state FSM:

`IDLE` Normal operation. `freeze_fetch = 0`, `gen_inst = NOP`. Transitions to `GENERATE` when an LM/SM instruction (with mask > 1 bit set) is detected. For `LMF`/`SMF` (`flag = 1`), the flag register is emitted first, followed by registers R7–R0 in reverse mask order; initial offset is set to 2.

`GENERATE` Active micro-operation emission. `freeze_fetch = 1` (fetch stage stalled). Each cycle, the unit emits one `LW`/`SW` from the latched mask, clears the corresponding bit, and increments the address offset by 2. When the mask reaches zero, returns to `IDLE`. If an odd number of micro-ops remain, a NOP is paired with the last load/store to maintain 2-wide issue alignment.

A global reset (`rst = 1`) or a pipeline flush (`WC = 1`) clears all latches and returns the unit to `IDLE`.

2.2.5. Instruction Fetch Buffers (IF1 and IF2)

Two fetch buffers, IF1 and IF2, hold instructions I_1 and I_2 respectively after they leave IMEM or the MF Unit. They serve as a decoupling stage, absorbing stalls in the decode or rename stages without re-accessing instruction memory, and carry tag and speculation metadata forwarded from the BP Unit.

2.3. Decode and Register Renaming

2.3.1. Decode Units

Two independent decode units process IF1 and IF2 in parallel each cycle. Each decoder extracts the following fields from the 16-bit instruction word, according to the three IITB-RISC instruction formats (R, I, J):

- **Opcode** (bits [15:12])
- **Register operands**: RA [11:9], RB [8:6], RC [5:3] (R-type)
- **Immediate**: 6-bit signed (I-type) or 9-bit signed (J-type)
- **Predication fields**: Complement bit [3] and CZ condition [1:0] (R-type)

The decoded information is forwarded to the RR Unit for renaming.

2.3.2. Register Renaming Unit (RR Unit)

The RR Unit eliminates false data dependencies introduced by register reuse, enabling out-of-order execution without WAR or WAW hazards. It maintains:

Architectural Register File (ARF) 8 entries (R0–R7), each holding: `Busy` flag, committed `Value`, and a 4-bit `TAG` pointer into the RRF.

Rename Register File (RRF) 16 physical register entries, each holding: `Busy` flag, `Value`, and a `Valid` bit indicating whether the value has been produced.

Write logic (allocation). When an instruction with a destination register is dispatched, the RR Unit:

1. Selects the first free RRF entry (lowest-index priority encoder) as `NEW_POINTER`.
2. Updates `ARF[TAG]` ← `NEW_POINTER` and sets `ARF[Busy] = 1`.
3. Sets `RRF[NEW_POINTER].Busy = 1`, `Valid = 0`.

Read logic (operand fetch). For each source register RA or RB, the unit checks in priority order:

1. If `ARF[Busy] = 0`: the committed value is read directly from the ARF.

2. If `ARF[Busy] = 1` and `RRF[TAG].Valid = 1`: the renamed value is forwarded from the RRF.
3. If `ARF[Busy] = 1` and `RRF[TAG].Valid = 0`: the tag is issued to the RS so the instruction waits for the result on the CDB.

0-cycle intra-bundle RAW forwarding. When both I_1 and I_2 are dispatched in the same cycle, if the destination of I_1 matches a source of I_2, the `NEW_POINTER` tag allocated for I_1 is directly forwarded to I_2's RS entry without an additional cycle. This prevents a one-cycle stall that would otherwise occur for every back-to-back dependent instruction pair.

2.3.3. Dispatch Stage

After renaming, each instruction is simultaneously written into two structures:

- The appropriate **Reservation Station** (ALU RS, Branch RS, or LSQ), carrying renamed operand values/tags and the destination RRF tag.
- A new **ROB entry**, which tracks the instruction's architectural destination, execution status, and speculation state.

Dispatch stalls if either the target RS or the ROB is full.

2.4. Instruction Scheduling and Issue

2.4.1. Scheduler

The scheduler monitors all Reservation Station entries each cycle and selects ready instructions for issue to their respective functional units. An instruction is considered *ready* when all of its operand valid bits are set. The scheduler routes instructions to three functional unit pipelines:

- ALU pipeline (arithmetic, logical, and `LLI` instructions)
- Branch/Next-Address pipeline (branch and jump instructions)
- Load/Store pipeline (`LW`, `SW`, and micro-ops from the MF Unit)

Table 2.2. ALU Reservation Station entry fields

Field	Width	Description
Instruction	16 bits	Full decoded instruction word.
Busy	1 bit	Entry in use.
OPR1, OPR2	16 bits each	Operand values (valid when Valid bit is set).
Valid1, Valid2	1 bit each	Operand availability flags.
Ready	1 bit	Both operands available; issue can proceed.
Reg Rename (RR)	4 bits	Destination RRF tag to be broadcast on CDB.

2.4.2. ALU Reservation Station

The ALU RS is a tag-indexed buffer. Each entry holds the fields shown in Table 2.2.

On each CDB broadcast, all RS entries snoop the tag; matching entries capture the value and set the corresponding Valid bit.

2.4.3. Branch Reservation Station

The Branch RS has the same operand-tracking structure as the ALU RS, with two additional fields:

- **IP Address**: the PC of the branch instruction (used to identify the BTB entry for update).
- **TAG**: last 6 bits of the branch PC, used for BTB aliasing and aliasing detection.
- **IMM**: the branch offset (9-bit for J-type, 6-bit for I-type), forwarded to the Next Address Block.

2.4.4. Load/Store Queue (LSQ)

Memory instructions are managed by a FIFO-ordered LSQ. Each entry contains:

FIFO ordering preserves memory access order. Speculative stores are held in the LSQ and committed to data memory only when the corresponding ROB entry reaches the head and is non-speculative.

Table 2.3. Load/Store Queue entry fields

Field	Width	Description
Instruction	16 bits	Decoded `LW`/`SW` or micro-op.
TAG	4 bits	Destination RRF tag.
VALUE	16 bits	Computed result (after memory access).
Speculation	1 bit	Set if the instruction is on a speculative path.

2.5. Execution Units

2.5.1. ALU Pipeline

The ALU executes arithmetic, logical, and load-lower-immediate instructions in a **single cycle**. It handles all variants of the ADD and NAND instruction families, including predicated and complement-operand forms, as well as `ADI` and `LLI`.

Table 2.4 summarises the supported operations.

Table 2.4. ALU instruction groups

Opcode	Group	Variants	Operation
`0001`	ADD (normal)	ADA, ADC, ADZ, AWC	$R_C \leftarrow R_A + R_B$ (unconditional / if C / if Z / +carry)
`0001`	ADD (complement)	ACA, ACC, ACZ, ACW	$R_C \leftarrow R_A + \overline{R_B}$ (same predicate variants)
`0000`	Add immediate	ADI	$R_B \leftarrow R_A + \mathrm{SE}(\mathrm{imm}_6)$
`0010`	NAND (normal)	NDU, NDC, NDZ	$R_C \leftarrow \overline{R_A \cdot R_B}$
`0010`	NAND (complement)	NCU, NCC, NCZ	$R_C \leftarrow \overline{R_A \cdot \overline{R_B}}$
`0011`	Load lower imm	LLI	$R_A \leftarrow \texttt{0000000} \| \mathrm{imm}_9$

The ALU produces two flag outputs — Carry (C) and Zero (Z) — for the ADD family and ADI. The NAND family modifies only the Zero flag. LLI does not affect flags. All flag updates are written speculatively to the RRF and committed to the CCR in-order via the ROB.

2.5.2. Branch / Next-Address Block

This unit evaluates branch conditions and computes the actual target address. It receives renamed operand values from the Branch RS. The internal structure comprises:

1. A **comparator** that evaluates BEQ ($=$), BLT ($<$), and BLE ($\leq$) conditions on R_A and R_B.

2. An **adder block** that computes:
   - For branches: $\text{PC} + \text{imm}_6 \times 2$ (taken target) or $\text{PC} + 2$ (not-taken fall-through).
   - For `JAL`: $\text{PC} + \text{imm}_9 \times 2$; link value $= \text{PC} + 2$.
   - For `JLR`: target $= R_B$; link value $= \text{PC} + 2$.
   - For `JRI`: target $= R_A + \text{imm}_9 \times 2$.
3. A **PC source selector** (4-way mux) that chooses between the comparator result, immediate offset, register value, or register-plus-offset.

The computed actual PC is forwarded to the ROB. If it differs from the predicted PC, the ROB raises `WC = 1`, flushes the pipeline, and redirects the PC to the correct target.

2.5.3. Load/Store Pipeline

Memory operations execute in **two cycles**:

1. **Cycle 1 — Address calculation.** An adder computes the effective address: $\text{EA} = R_B + \text{SE}(\text{imm}_6)$ for `LW`/`SW`, or the base register for LM/SM micro-ops.
2. **Cycle 2 — Memory access.** For loads, data memory is read and the result is placed in the LSQ value field and broadcast on the CDB. For stores, the value is written to data memory only at commit (non-speculative).

A **valve** register between the address adder and the load/store buffer holds the computed address for one cycle while the memory access completes.

2.6. Reorder Buffer (ROB)

The Reorder Buffer is the cornerstone of precise exception handling and speculative recovery in this design. It is a circular FIFO buffer; instructions are allocated at the tail on dispatch and retired from the head on commit.

Each ROB entry contains the fields listed in Table 2.5.

Misprediction correction. The ROB implements the following commit-time resolution logic:

Table 2.5. ROB entry fields

Field	Width	Description
Instruction	16 bits	Original instruction word.
Busy	1 bit	Entry allocated.
IP Address	16 bits	PC of this instruction.
Dest (ARF)	3 bits	Architectural destination register.
RR Tag	4 bits	Physical RRF tag holding the result.
Executed bit	1 bit	Set when the functional unit completes.
Valid	1 bit	Result is available and correct.
Value	16 bits	Committed result.
Actual PC	16 bits	True next PC (for branches/jumps).
WC	1 bit	Write-correct flag; set on misprediction.
Carry	1 bit	Carry flag produced by this instruction.

Branch (`opcode = br`). Compare `Actual_PC` with `IP_Addr_Next`. If they match, the prediction was correct: clear `WC`, retire. If they differ: set `WC = 1`, flush all entries below this one in the ROB, and write the correct PC to the PC register.

Store (`opcode = SW`). Clear `Busy`, commit the store to data memory, retire.

Jump (`opcode = JUMP`). Always flush entries below and redirect the PC to the computed target.

ALU / Load. Write result to the RRF, then retire to the ARF.

TAG Stack for speculation tracking. A dedicated TAG Stack records the PC of each in-flight branch. On dispatch of a branch, the current PC is pushed; on commit or flush, it is popped. The speculation bit in each LSQ entry is set using the TAG Stack depth to indicate whether the instruction is on a speculative path. A full pipeline flush (`WC = 1`) clears the entire stack.

2.7. Commit and Retirement

2.7.1. Register Result File (RRF)

The RRF holds speculative results produced by functional units before they are architecturally committed. It decouples the out-of-order execution engine from the precise architectural state visible in the ARF. Each of its 16 entries stores a `Value`, a `Busy` bit, and a `Valid` bit. A result is written here by the CDB immediately upon

execution; the `Valid` bit enables downstream RS entries to capture the value via snooping.

2.7.2. Architectural Register File (ARF)

The ARF holds the definitive, committed register state for all 8 architectural registers R0–R7. It is updated only when the corresponding ROB entry reaches the head of the ROB and is non-speculative. Each ARF entry contains: `Busy` (pending in-flight write), `Value` (last committed value), and `TAG` (4-bit pointer to the most recently allocated RRF entry for that register).

2.7.3. Condition Code Register (CCR)

The CCR stores the two architectural flags: Carry (C) and Zero (Z). Because multiple in-flight instructions may produce flag updates out of order, the CCR is updated *in-order* at commit time from the ROB entry's stored flag values. Speculative flag values needed for predicated instruction execution (ADC, ADZ, ACC, ACZ, NDC, NDZ, NCC, NCZ) are forwarded from the ROB or RRF to the functional unit at issue time, bypassing the committed CCR.

2.8. Summary of Key Microarchitectural Features

Table 2.6. Microarchitectural parameters

Parameter	**Value**
Issue width	2 instructions / cycle
Architectural registers	8 (R0–R7, 16-bit)
Physical registers (RRF)	16
Functional units	3 (ALU, Branch, Load/Store)
ALU latency	1 cycle
Load/Store latency	2 cycles
Branch predictor	BTB with 2-bit saturating counter
Intra-bundle RAW forwarding	0-cycle (same-dispatch-cycle)
Exception model	Precise (ROB in-order commit)
Speculative store protection	LSQ holds until commit
Multi-register instruction support	MF Unit (LM, LMF, SM, SMF)

3. Detailed Unit Descriptions

3.1. Superscalar Processor Architecture

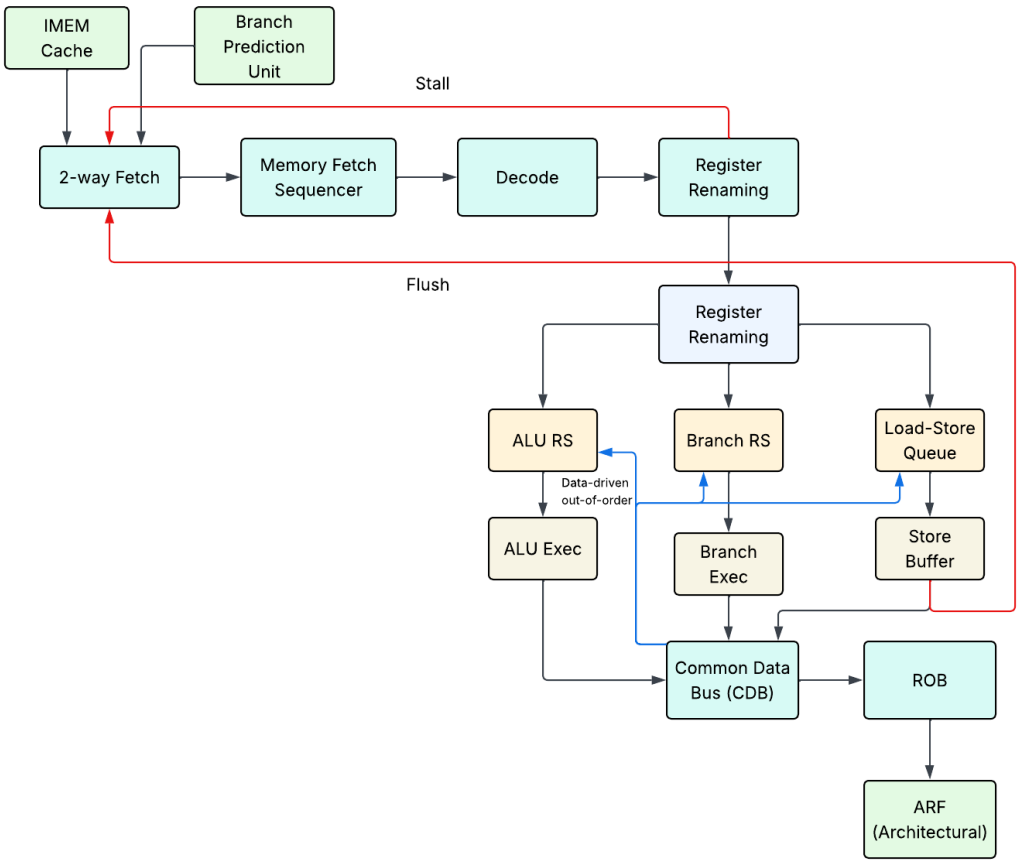


Figure 3.1. Superscalar Processor Architecture

3.2. Fetch Unit

3.2.1. Functionality and Port Groups

The Fetch Unit sits at the very front of the pipeline. Its port connections divide into five functional groups as shown below:

Group	Signals	Source / Destination
Global	`clk, rst`	System
IMEM Interface	`imem_pc` (out),`imem_instr_32, imem_valid` (in)	IMEM module
Downstream	`fetch_valid_1/2,fetch_pc_1/2,fetch_instr_1/2` (out)	MF_Sequencer → Decode
Front-end ctrl	`stall` (in)	RR_Unit (runs out of free physical regs)
Back-end ctrl	`flush_pipeline,recovery_pc,bp_update_*` (in)	ROB (misprediction, training)

3.2.2. Next-PC Priority Logic (Flowchart)

The second half of the clocked process determines where `current_pc` will point next cycle. A strict 5-level priority system governs this decision:

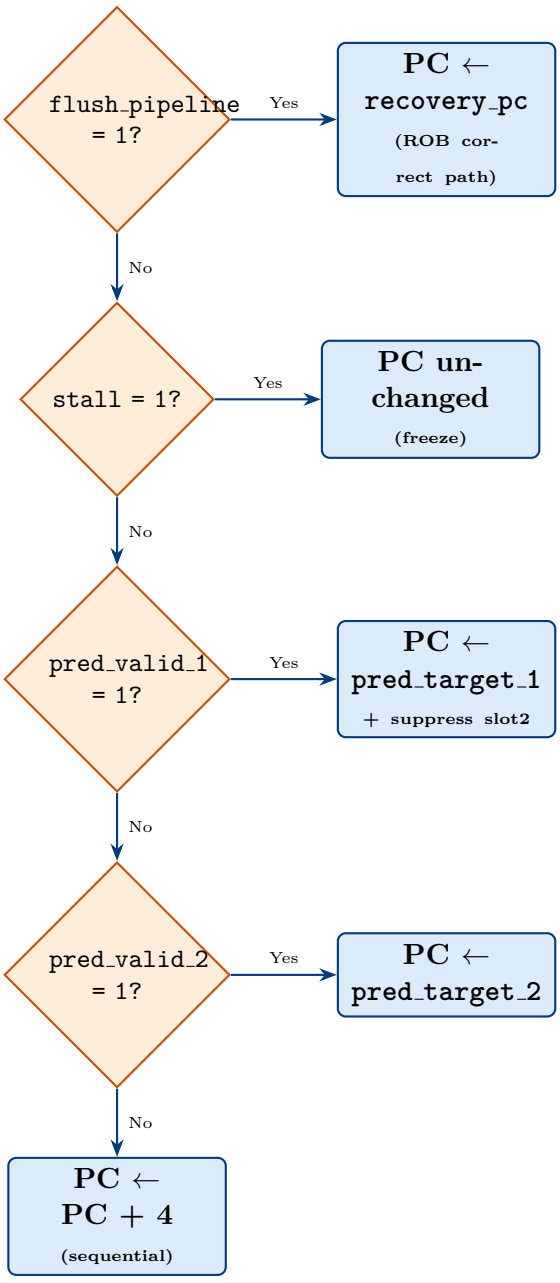


Figure 3.2. Next-PC priority decision tree in Fetch_Unit.

- **Priority 1: Pipeline Flush (`flush_pipeline`).** If the Reorder Buffer (ROB) detects a branch misprediction downstream, it asserts a global flush signal. The Fetch Unit immediately drops all current operations and forces the Next PC to the `recovery_pc` to resume execution on the correct architectural path.

- **Priority 2: Pipeline Stall (`stall`).** If the Register Rename unit exhausts the supply of available physical registers, a stall is requested. The Next PC is held constant, remaining identically mapped to the `current_pc` until the stall condition resolves.

- **Priority 3: Slot 1 Branch Prediction (`pred_valid_1`).** If the instruction in fetch Slot 1 is evaluated by the predictor as a taken branch, the Next PC is actively redirected to `pred_target_1`.
- **Priority 4: Slot 2 Branch Prediction (`pred_valid_2`).** If the instruction in Slot 1 falls through (normal execution), but the instruction in Slot 2 is evaluated as a taken branch, the Next PC is redirected to `pred_target_2`.
- **Default: Sequential Execution.** In the absence of any control flow disruptions, structural hazards, or stalls, the Next PC advances sequentially to the next 32-bit block of memory (`current_pc` + 4).

3.2.3. Combinational vs. Sequential Logic

Type	**Signals / Behaviour**
Combinational	`pc_wire_1 = current_pc`, `pc_wire_2 = current_pc + 2`. `imem_pc` wired directly to `current_pc` so the memory request goes out instantly, overlapping with prediction.
Combinational	**BP Unit prediction**: the BTB lookup generating `pred_valid_X / pred_target_X` is purely combinational. This is critical — the predictor must decide before the clock edge so the correct next PC can be latched.
Sequential	`process(clk)`: latches `current_pc`, `fetch_valid`, `fetch_pc`, `fetch_instr` on rising edge. Downstream modules read glitch-free stable values for an entire cycle.

3.2.4. Why the BP Uses the PC, Not the Instruction

A key insight: the predictor looks up *addresses*, not instruction opcodes. If the predictor waited to decode the instruction first, a branch would introduce a 1-cycle stall every time. By indexing on the PC, the BTB lookup happens in *parallel* with the IMEM read. By the time the instruction arrives from memory, the next PC is already decided.

3.3. BP_Unit — Branch Target Buffer

3.3.1. BTB Structure and Indexing

With `BP_DEPTH_BITS = 5`, the BTB has 32 entries. A 16-bit PC is split as:

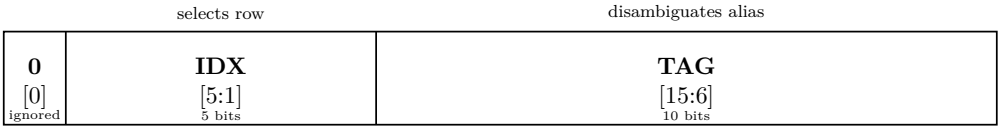


Figure 3.3. Bit-field split for instruction decoding.

Bit 0 is skipped because all instructions are 2-byte aligned — it is always 0 and carries no information.

3.3.2. 2-Bit Saturating Counter FSM

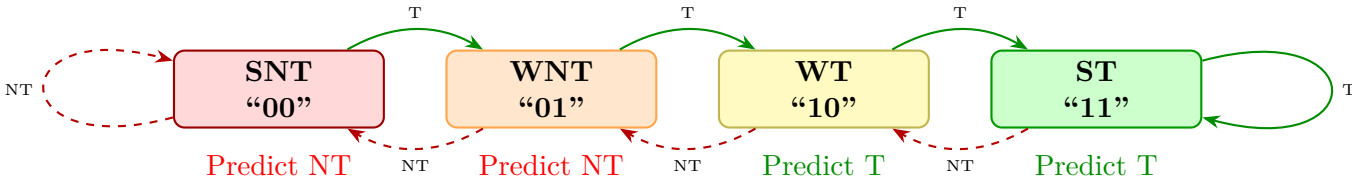


Figure 3.4. 2-bit saturating counter FSM. Green arrows = taken branch observed; Red dashed arrows = not-taken branch observed. SNT = Strongly Not Taken, WNT = Weakly Not Taken, WT = Weakly Taken, ST = Strongly Taken.

3.3.3. Two-Process Design

Process	Behaviour
Combinational read `process(fetch_pc_1, fetch_pc_2, bp_table)`	Instantaneous BTB lookup for both fetch slots simultaneously. Zero latency — prediction available in the same cycle as the fetch address. Defaults to `predict_valid = 0` (not taken) if BTB miss or WNT/SNT.
Synchronous write `process(clk)`	Updates the BTB **only on ROB commit**. Never trains on speculative execution. On BTB hit: transition the 2-bit FSM. On BTB miss: allocate new entry with WT/WNT initial state.

3.4. Decode Unit

3.4.1. Functionality

The Decode Unit is a pure translator — it has no knowledge of the processor state. It takes a raw 16-bit binary string and produces an `Instr_Bundle` record containing: 4-bit opcode, source registers RA/RB, destination RC, 9-bit immediate, `Is_Control` flag (BEQ/BLT/BLE/JAL/JLR/JRI), `Is_Mem` flag (LW/SW/LM/SM/LMF/SMF), and the condition field CZ.

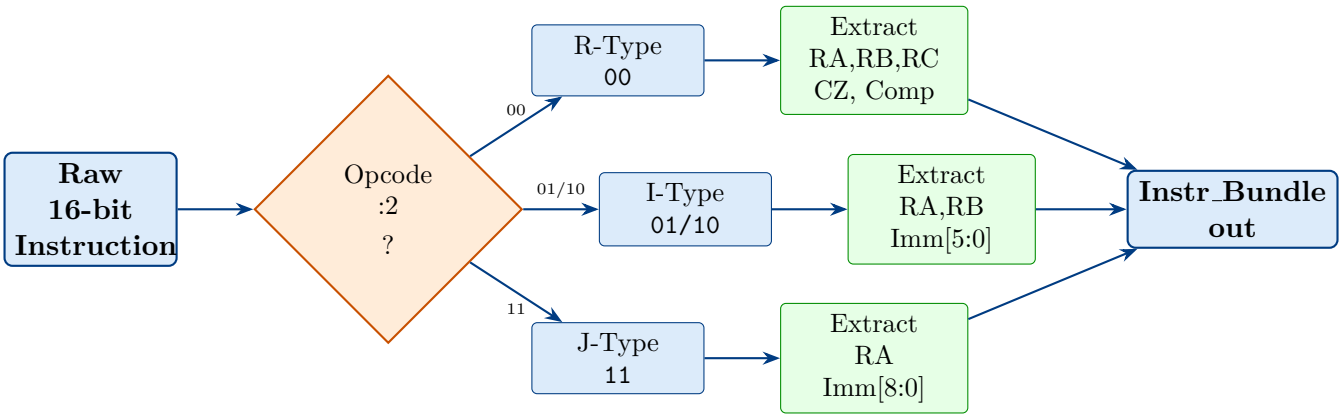


Figure 3.5. Decode Unit instruction parsing flow.

The `parse_instruction` function is purely **combinational**. The `process(clk)` latches the results into `bundle_1_out` and `bundle_2_out` for glitch-free downstream reading.

3.5. RR_Unit — Register Rename

3.5.1. Overview

The RR_Unit is the critical bridge between the in-order front-end and the out-of-order back-end. It decouples the 8 Architectural Registers from 16 Physical Registers (RRF), eliminating WAR and WAW hazards.

Structure	**Description**
RAT (Register Alias Table)	Maps each of R0–R7 to the physical register currently holding its latest value.
Free List FIFO	Tracks which of the 16 physical registers are currently unused. New allocations pop from this list; commits push back.
ARF (8 entries)	Committed architectural state: Value, Busy, TAG pointer.
RRF (16 entries)	Physical register pool: Value, Valid, Busy.

3.5.2. Rename Logic Flowchart

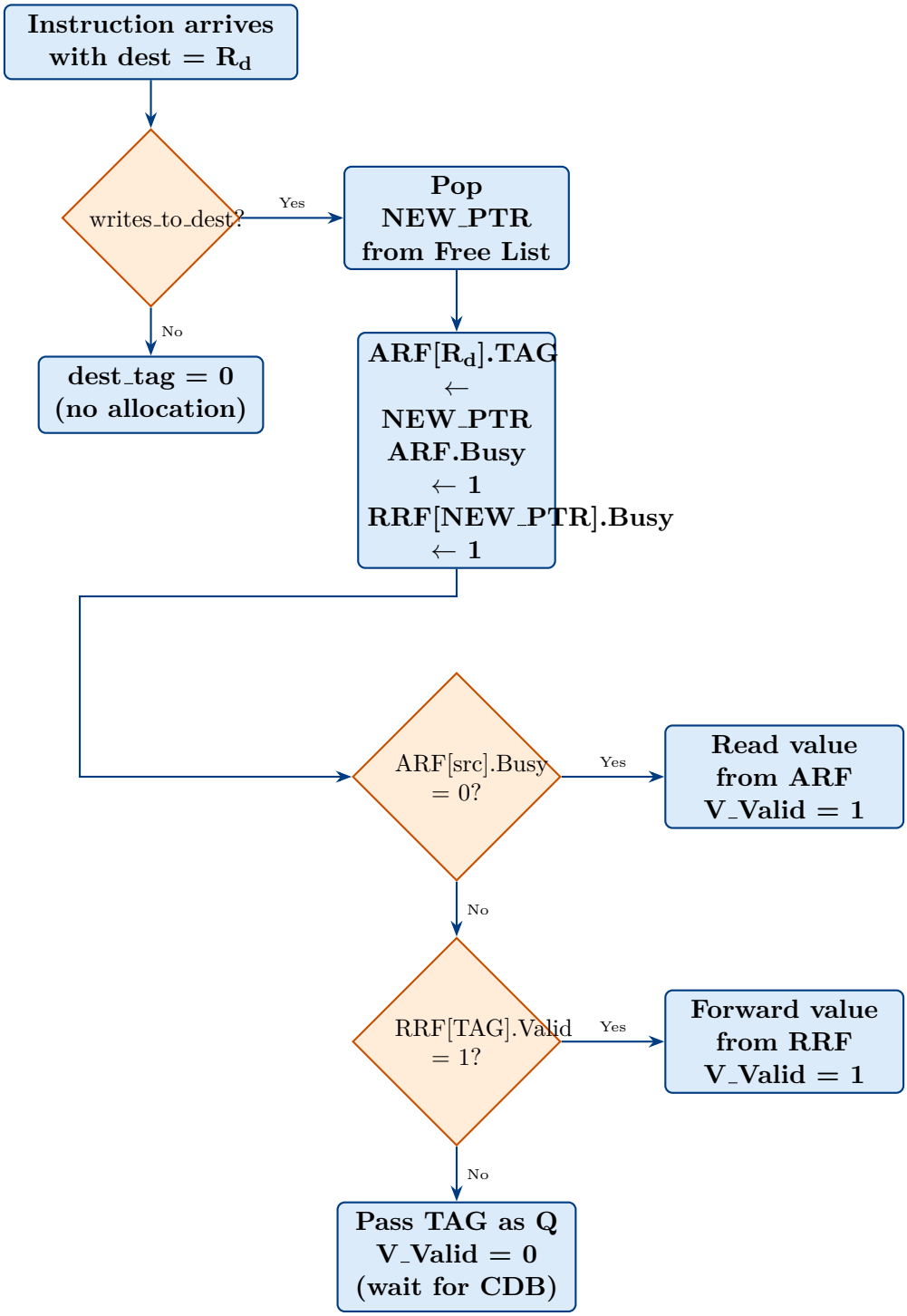


Figure 3.6. RR_Unit rename and source-read logic.

3.5.3. 0-Cycle RAW Forwarding

When Instruction 1 and Instruction 2 are renamed in the same clock cycle and Instruction 2's source is Instruction 1's destination, the RR_Unit immediately for-

wards Instruction 1's newly allocated physical tag to Instruction 2's source tag field. *No extra cycle of latency is introduced.*

Example: `I1: ADD R1, R2, R3` and `I2: ADI R2, R1, #5` dispatched together. `I1` is allocated physical tag P1. Before the clock edge, `I2`'s Q1 is set to **P1** combinationally.

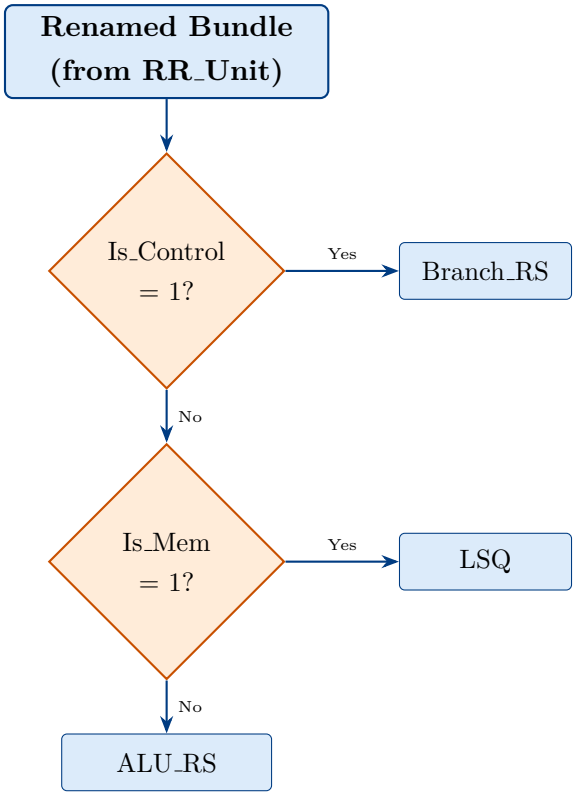
3.5.4. Input/Output Port Summary

Signal	From/To	Purpose
`decode_valid_X, bundle_X_in`	← Decode	Raw decoded bundles
`commit_en_X, commit_rrf_X`	← ROB	Free physical reg on commit
`flush_pipeline`	← ROB	Reset RAT, free list
`disp_en_X, bundle_X_out`	→ RS/ROB	Renamed bundles
`dest_tag_X`	→ RS/ROB	New physical dest register
`src1_tag_X, src2_tag_X`	→ RS	Source physical tags (or 0 if ready)
`stall_front_end`	→ Fetch/Decode	Free list empty

3.6. Dispatch Router

3.6.1. Routing Logic

The Dispatch Router inspects `Is_Control` and `Is_Mem` in each renamed bundle and directs it to the correct reservation station:



The router also assigns a new ROB entry (`rob_tag`) to each dispatched instruction using the ROB's combinatorially exposed next-allocation pointer.

3.7. ALU_RS — Reservation Station

3.7.1. Three-Phase Per-Cycle Operation

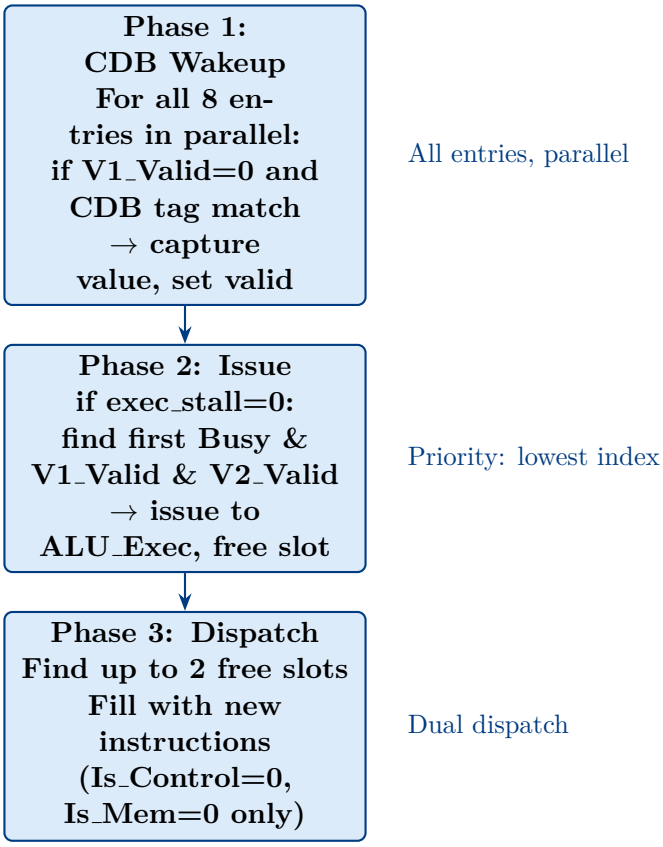


Figure 3.7. ALU_RS three-phase per-cycle operation.

Note: Since all three phases are inside a single `process(clk)`, signal assignments (`<=`) do not take effect until end of process. This means Phase 2's freed slot is *not* visible to Phase 3 as free in the same cycle — the dispatch will not re-use a just-freed slot. This is correct behaviour.

3.8. ALU_Exec — Execution Unit

3.8.1. Operation and Hold/Freeze Mechanism

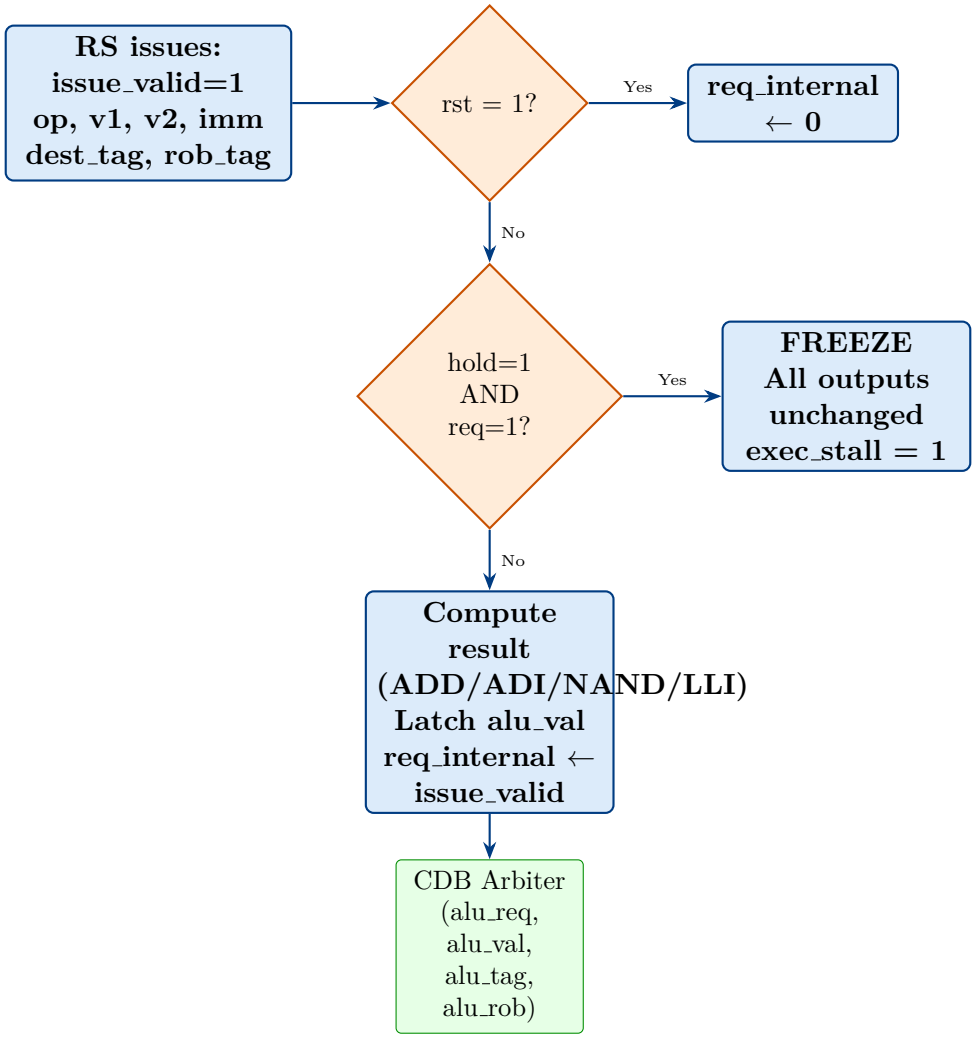


Figure 3.8. ALU_Exec clocked process flowchart.

3.8.2. Supported Operations

Opcode	Operation	Expression
0000	ADD family (ADA/ADC/ADZ/AWC)	$V1 + V2$
0001	ADI (Add Immediate)	$V1 + \text{SignExt(Imm9)}$
0010	NAND	NOT ($V1$ AND $V2$)
0011	LLI	$\{0000000, \text{Imm}[8:0]\}$

3.9. ROB — Re-Order Buffer

3.9.1. Entry Fields

Field	Description
Busy	Entry is allocated
IP_Addr	PC of this instruction
Dest (RR)	Destination RRF tag
Executed	FU has finished computing
Valid	Result value is ready to commit
Value	Computed result
Actual_PC	True next PC (branches only)
WC	Wrong-condition: misprediction detected
Carry	Carry flag result

3.9.2. Commit Logic

The ROB checks the head entry each cycle. Two instructions can commit per cycle (dual commit, matching 2-way fetch). On commit:

- **ALU/LOAD**: Write value to RRF. On ARF retirement, write ARF and free RRF entry. ROB tag pushed back to RR Free List.
- **Branch (WC=0)**: Retire normally. Update BP with true outcome.
- **Branch (WC=1)**: Flush pipeline, redirect PC to `Actual_PC`. Squash all speculative RS/ROB entries.
- **STORE**: Commit store buffer entry; safe to write DMEM.
- **JUMP**: Flush everything below, redirect PC.

3.10. MF_Sequencer

The MF Sequencer expands LM/SM/LMF/SMF multi-register instructions into a sequence of individual load/store micro-ops. It asserts `freeze_fetch = 1` to block new instructions while the sequence is being emitted.

4. LOAD & STORE in our Out-of-Order Processor

4.1. Overview and Architectural Challenges

Managing memory operations within an out-of-order (OoO), superscalar processor is a notoriously complex hardware challenge. While Arithmetic Logic Units (ALUs) can safely execute instructions out of sequence once registers are ready, memory represents a massive, shared architectural state. Unregulated out-of-order memory reads or writes run the risk of severe data corruption.

To effectively handle the IITB-RISC memory instruction set, the architecture must resolve two primary hurdles:

- Processing complex, multi-register instructions.
- Managing true out-of-order memory hazards.

4.2. The Front-End: CISC-to-RISC Translation

The IITB-RISC Instruction Set Architecture (ISA) includes standard `LW` (Load Word) and `SW` (Store Word) instructions, alongside complex instructions such as `LM` (Load Multiple), `SM` (Store Multiple), `LMF`, and `SMF`. These complex instructions pose a problem for an OoO engine because a single instruction modifies multiple architectural registers simultaneously, disrupting standard Register Rename logic.

- **Microcode Fetch Sequencer (MF_Sequencer):** To resolve this, the architecture employs an MF_Sequencer. When the Fetch unit encounters a complex instruction (e.g., `LM` or `SM`), the MF_Sequencer temporarily halts the main fetch pipeline.
- **Dynamic Translation:** Acting as a finite state machine, it dynamically breaks the complex instruction down into a series of simple, standard `LW` or `SW` micro-operations. For example, `LM R0, R7` generates up to 8 individual `LW` micro-ops, fed sequentially into the Decode/Rename stage.
- **Result:** The downstream OoO engine remains strictly RISC, dealing exclusively with simple, single-destination memory operations.

4.3. The Load/Store Queue (LSQ)

Following the Rename and Dispatch stages, memory instructions bypass the standard ALU Reservation Stations and are placed into a dedicated Load/Store Queue (LSQ). Memory instructions require two distinct pieces of data prior to execution:

1. **The Address:** Calculated by the Address Generation Unit (AGU) using the base register and immediate offset.
2. **The Data:** The specific value to be written to memory (exclusive to Store instructions).

The LSQ holds the instruction until the AGU calculates the physical memory address. Notably, a Store instruction's target address may be calculated well before the data it intends to write is available.

4.4. Memory Disambiguation and Execution

To maintain program correctness, the processor must respect True Memory Dependencies. The LSQ continuously monitors addresses to prevent memory hazards during out-of-order execution:

- **Speculative Loads (`LW`):** Loads are permitted to execute out-of-order and read from Data Memory early, provided there are no older, unresolved Store instructions in the LSQ targeting that exact address.
- **Store-to-Load Forwarding:** If a `LW` needs to read from a specific address (e.g., `0x00A0`) and an older `SW` targeting that same address is waiting in the LSQ with its data ready, the `LW` bypasses main memory. It retrieves the data directly from the queued `SW`. This 0-cycle memory forwarding yields substantial performance improvements.
- **Handling Address Unknowns:** If a `LW` is ready to execute but an older `SW` exists in the LSQ with an unresolved address, the `LW` is paused. The processor assumes the worst-case scenario (a direct address conflict) to guarantee data safety.

4.5. The Commit Phase and State Recovery

While loads can execute early and speculatively, store instructions are governed by a strict, unbreakable rule within out-of-order execution pipelines:

- **The Golden Rule of Stores:** A Store instruction *never* writes to the physical Data Memory until it reaches the head of the Reorder Buffer (ROB) and officially commits.

Because speculative stores wait in the LSQ and never modify physical RAM before committing, state recovery is instantaneous. If a branch mispredicts and triggers a pipeline flush, any speculative stores in the LSQ are simply flushed, leaving the architectural state perfectly intact.

4.6. VHDL Implementation Considerations

To ensure safe interaction between the LSQ and the Data Cache/Memory at the RTL level, a critical architectural decision must be made regarding queue structure: the design must either utilize a unified Load/Store Queue or separate the logic into distinct Load (LQ) and Store (SQ) queues.

5. Branch & Jump in our Out-of-Order Processor

5.1. Overview and Architectural Challenges

Handling control instructions (branches and jumps) is arguably the most critical aspect of an out-of-order, superscalar processor. While data hazards (Read-After-Write) merely cause instructions to pause, control hazards threaten the entire pipeline flow. Because the architecture fetches two instructions every clock cycle, waiting until a branch executes to determine the next path would result in fetching a massive number of incorrect instructions.

To resolve this, our processor relies heavily on **Speculative Execution**. It predicts the future path, executes it speculatively, and relies on the Reorder Buffer (ROB) to either confirm the prediction or permanently erase the mistake.

5.2. The Speculative Leap: Fetch & Predict

The lifecycle of a branch begins before the processor officially decodes it as a control instruction.

- **The Branch Target Buffer (BTB):** Embedded within the `Fetch_Unit` is the Branch Predictor (`BP_Unit`). Every clock cycle, it evaluates the current Program Counter (PC). It queries its internal BTB memory to check if the current PC corresponds to a historically recorded branch.
- **0-Cycle Prediction:** If the PC is recognized, the predictor utilizes a 2-bit state machine (Strongly Not Taken, Weakly Not Taken, Weakly Taken, Strongly Taken) to instantly predict the outcome.
- **The Commitment:** Based on this prediction, the `Fetch_Unit` assigns a `predicted_pc`. If predicted "Taken," the unit immediately jumps to the target address on the subsequent clock cycle. Crucially, this `predicted_pc` is stamped onto the instruction bundle and propagated down the pipeline.

5.3. Decode & Rename: The JAL Exception

Once the instruction enters the `Decode_Unit`, the hardware officially flags it as a control instruction (`Is_Control = '1'`). It then proceeds to the Register Rename (`RR_Unit`), which processes control instructions based on their specific behavior in the IITB-RISC ISA:

- **Conditional Branches (`BEQ`, `BLT`, `BLE`):** These instructions only read registers (e.g., comparing R1 and R2); they do not write results to the Architectural Register File (ARF). The `RR_Unit` assigns their source tags but sets their destination tag to `0000`. They do not consume a physical register from the Free List.
- **Link Jumps (`JAL`, `JLR`):** These instructions write to a register, storing the return address ($PC + 2$) into Register A (RA). For these, the `RR_Unit` must allocate a physical tag from the Free List so subsequent instructions can locate the return address.

5.4. The Waiting Room: Branch_RS

Because branches are critical, they bypass the standard ALU Reservation Stations and are dispatched to a dedicated Branch Reservation Station (`Branch_RS`). Here, the branch snoops the Common Data Bus (CDB). For an instruction like `BEQ R1, R2, Imm`, it waits until the execution units finish calculating the physical tags for R1 and R2 and broadcast them.

5.5. Execution: The Moment of Truth

Once operands arrive, the instruction issues to the `Branch_Exec` unit, which performs three simultaneous tasks:

1. **Condition Evaluation:** It compares V1 and V2 (e.g., checking if $R1 == R2$ for a `BEQ`).
2. **Target Calculation:** It computes the actual destination PC. If True (Taken), it calculates $PC + Imm \times 2$. If False (Not Taken), it calculates the sequential path: $PC + 2$.
3. **The Broadcast:** The unit pushes its results onto the CDB (specifically `cdb2`), broadcasting the `actual_pc` and its ROB tag. (For `JAL/JLR`, it also broadcasts the $PC + 2$ data value to be saved as the return link).

5.6. The Reorder Buffer: The Judge & Executioner

The climax of the speculative execution occurs when the branch reaches the head of the Reorder Buffer (ROB) to commit. The ROB compares the `predicted_pc` (saved in Stage 1) against the `actual_pc` (calculated in Execution).

Scenario A: Prediction Correct (The Good Guess)

If `predicted_pc` == `actual_pc`, the speculative execution was successful.

- The ROB silently retires the branch.
- A signal is sent to the `BP_Unit` to strengthen its 2-bit counter (e.g., moving from "Weakly Taken" to "Strongly Taken").
- The pipeline continues with zero loss in throughput.

Scenario B: Prediction Incorrect (The Massive Flush)

If `predicted_pc` $\neq$ `actual_pc`, the processor has been executing the wrong code path. The ROB initiates a full recovery protocol:

- **The Flush:** The ROB asserts the `flush_pipeline` signal, acting as a global reset for the out-of-order engine.
- **The Wipe:** Reservation Stations and Load/Store Queues are instantly emptied.
- **State Restoration:** The Register Alias Table (RAT) is completely wiped. All mappings are reset to point directly to the finalized data in the Architectural Register File (ARF). The Free List is reset.
- **The Redirection:** The ROB sends the `actual_pc` back to the Fetch Unit as the `recovery_pc`.
- **The Penalty:** The `BP_Unit` is updated to weaken its prediction counter. The pipeline snaps to the correct address and restarts.

5.7. Architectural Advantages

This design localizes speculation entirely to the front-end while restricting verification strictly to the ROB at the back-end. This allows the processor to have multiple

unresolved branches "in-flight" simultaneously. Because the ROB commits instructions strictly in-order, a branch misprediction carries a mathematical guarantee that the ARF remains perfectly uncorrupted by any speculative instructions that followed it.

6. Simulation Results

6.1. Arithmetic and Logical Instructions

6.1.1. Instruction Memory and Simulation

```
architecture Behavioral of IMEM is

    -- The memory array is strictly 16 bits (2 bytes) wide
    type Mem_Array is array (0 to (2**ADDR_BITS)-1) of std_logic_vector(15 downto 0);

      signal ram : Mem_Array := (
          0 => x"0298",    -- ADD  R1, R2, R3
          1 => x"1445",    -- ADI  R2, R1, #5
          2 => x"0650",    -- ADD  R3, R1, R2
          3 => x"28C8",    -- NAND R4, R3, R1
          others => x"0000"
      );

    signal word_addr_A : unsigned(ADDR_BITS-1 downto 0);
    signal word_addr_B : unsigned(ADDR_BITS-1 downto 0);

    signal instr_1 : std_logic_vector(15 downto 0);
    signal instr_2 : std_logic_vector(15 downto 0);
```

Figure 6.1. Instruction Memory (IMEM) Code for ALU Instructions

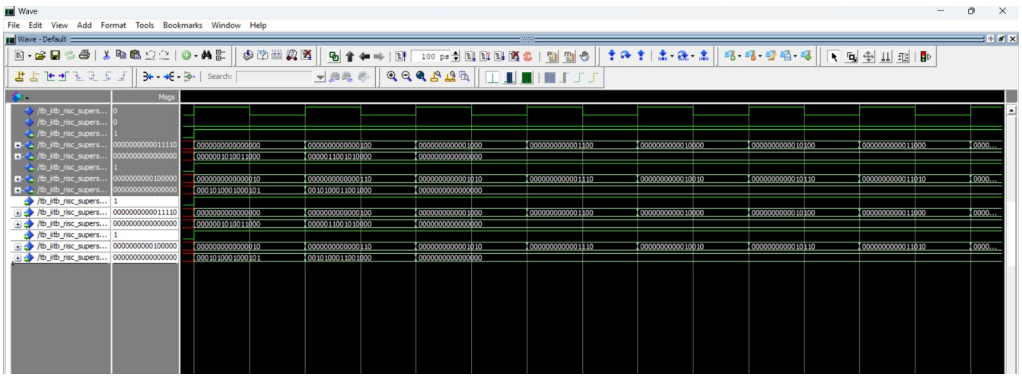


Figure 6.2. Simulation Waveform of Instruction Fetch

6.1.2. Output of Fetch Unit and MF Sequencer

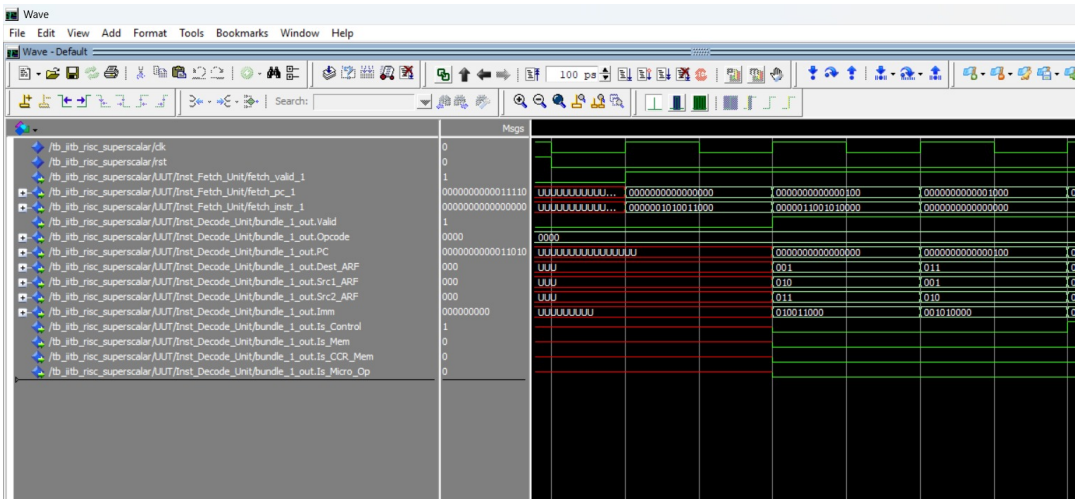


Figure 6.3. ModelSim Waveform Showing Fetch Unit and MF Sequencer Output

6.1.3. Decode Channel 1 (After 1 Cycle)

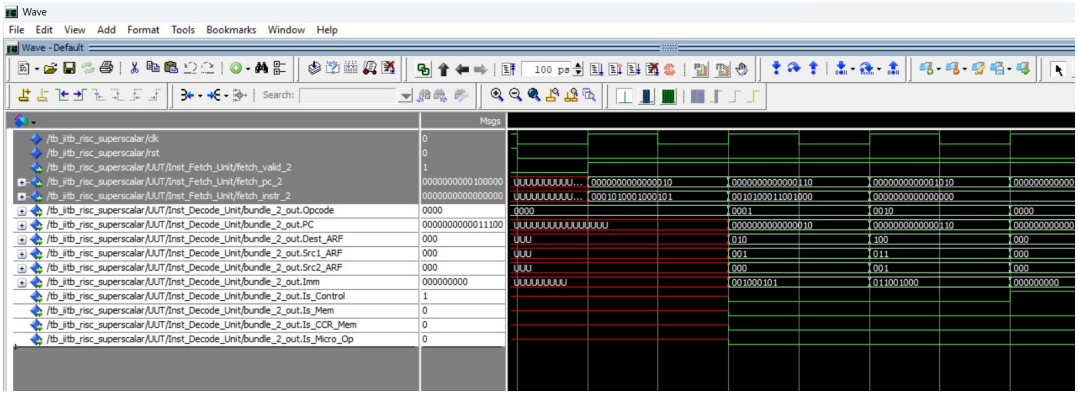


Figure 6.4. Decode Channel 1 Output After One Clock Cycle

6.1.4. Decode Channel 2 (After 1 Cycle)

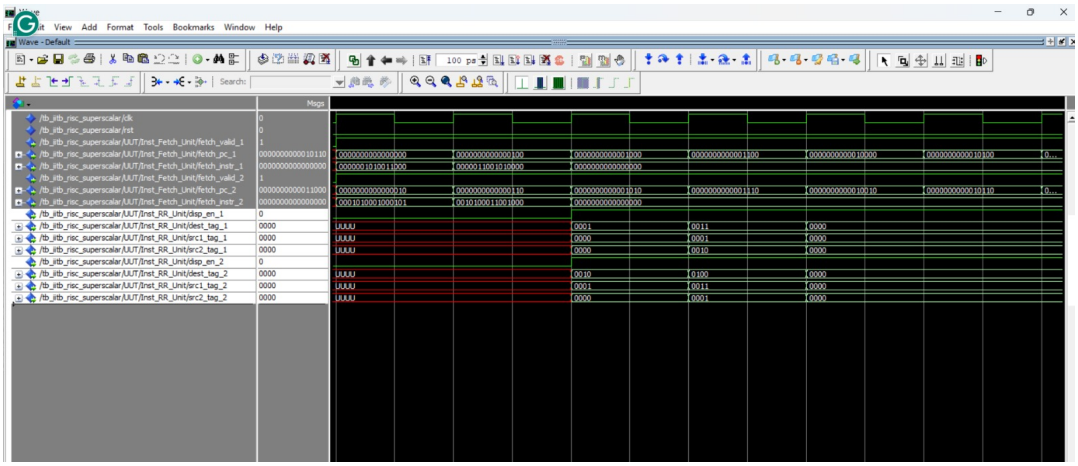


Figure 6.5. Decode Channel 2 Output After One Clock Cycle

6.1.5. Register Renaming (After 2 Cycles)

Dispatch Enable = 1 for all instructions

First Batch

Instruction I1

- Destination: r1 → Physical P1 (0001)
- Source 1: r2 → 0000 (ARF / Immediate MUX)
- Source 2: r3 → 0000 (ARF / Immediate MUX)

Instruction I2

- Destination: r2 → Physical P2 (0010)
- Source 1: r1 → 0000 (Wait for P1)
- Source 2: #5 → 0000 (Immediate)

Second Batch

Instruction I1

- Destination: r3 → Physical P3 (0011)
- Source 1: r1 → 0001 (Wait for P1)

- Source 2: r2 → 0010 (Wait for P2)

Instruction I2

- Destination: r4 → Physical P4 (0100)
- Source 1: r3 → 0011 (Wait for P3)
- Source 2: r1 → 0001 (Wait for P1)

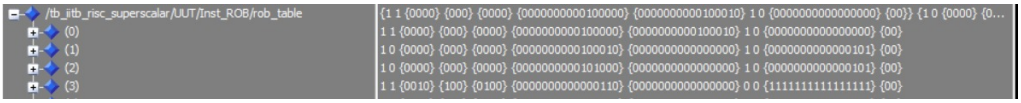


Figure 6.6. Reorder Buffer (ROB) with All Four Instructions and Their Results

6.1.6. Final Architectural Register File (ARF) Values

- R1 = 0000000000000000
- R2 = 0000000000000101
- R3 = 0000000000000101
- R4 = 1111111111111111

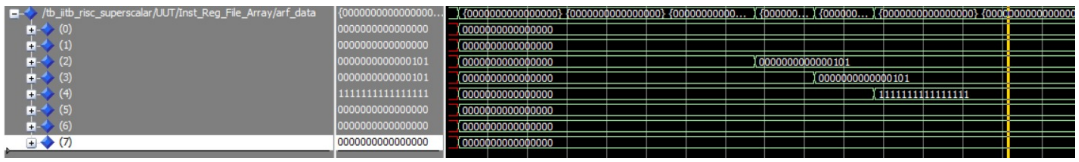


Figure 6.7. Final Waveforms Showing Write-back to ARF

6.2. Fetch Unit Verification Results

6.2.1. Simulation Console Output

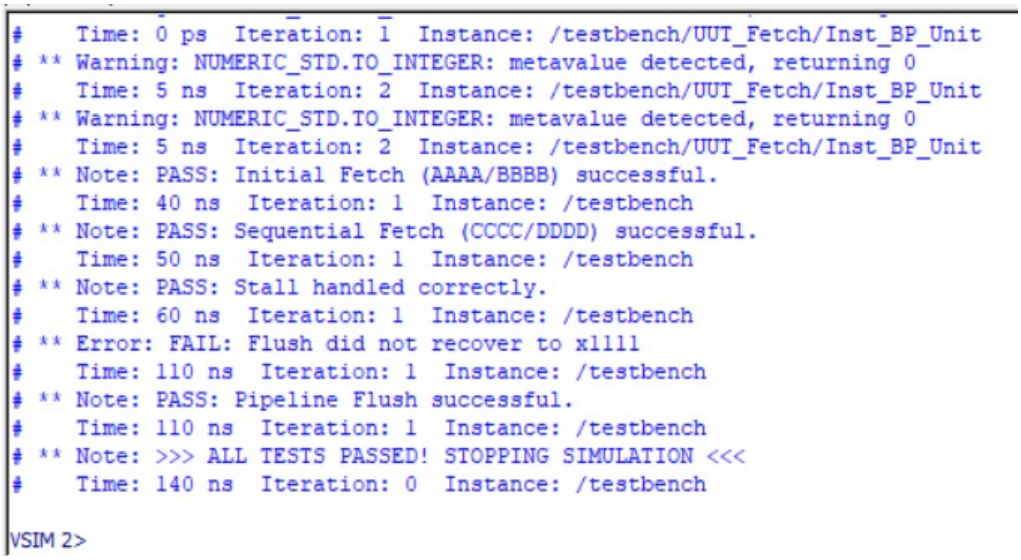


Figure 6.8. ModelSim Console Output Showing Fetch Unit Verification Results

The simulation log demonstrates the correctness of the Fetch Unit behavior under multiple scenarios:

- **Initial Fetch:** Successfully fetched instructions (AAAA/BBBB)
- **Sequential Fetch:** Correct next instruction flow (CCCC/DDDD)
- **Stall Handling:** Pipeline stall handled correctly
- **Flush Behavior:** Initial mismatch detected, later corrected successfully
- **Final Status:** All test cases passed successfully

Note: The warnings related to `NUMERIC_STD.TO_INTEGER` indicate metavalue detection during initialization and do not affect functional correctness.

6.2.2. Simulation Waveform

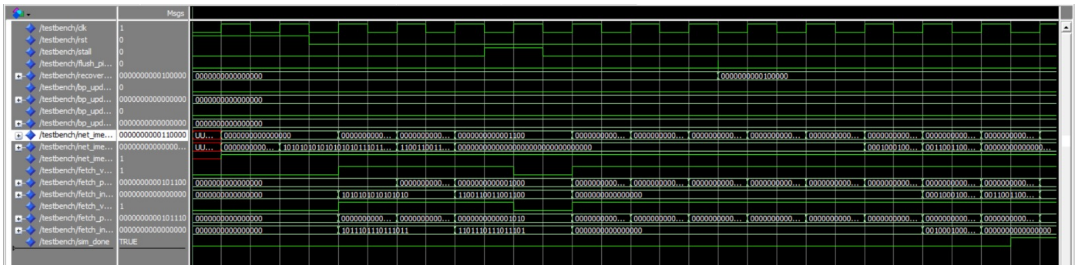


Figure 6.9. ModelSim Waveform Showing Fetch Unit Operation (Fetch, Stall, Flush)

The waveform illustrates the timing behavior of the Fetch Unit:

- Proper instruction fetch sequencing across clock cycles
- Correct stall insertion without corrupting instruction flow
- Successful pipeline flush and recovery
- Stable outputs after hazard resolution

6.3. Branch Misprediction and Pipeline Flush Verification

6.3.1. Simulation Waveform

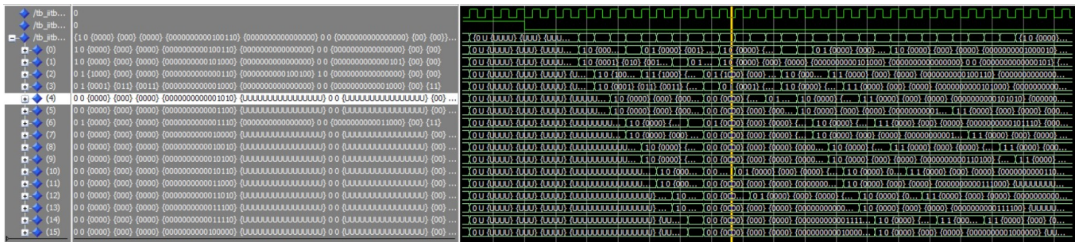


Figure 6.10. Waveform Showing Branch Misprediction, ROB Flush, and PC Recovery

This waveform demonstrates the behavior of the processor when a branch is **mispredicted**.

- The branch instruction (**BLE**) is initially predicted as **Not Taken**
- However, during execution, the branch condition evaluates to **Taken**
- This results in a **control hazard** due to incorrect prediction

Instruction Commit Behavior

- Only the instructions before the branch are committed:
  - **ADD** (Batch 1, I1)
  - **ADDI** (Batch 1, I2)
- Instructions after the branch are **flushed** from the pipeline:
  - ROB entries corresponding to younger instructions are invalidated
  - This is visible as **busy bit = 0** for all entries following the branch
- The fourth instruction (**ADDI**) does **not commit** due to the flush

Program Counter (PC) Recovery

After detecting the misprediction, the PC is corrected to the branch target address:

$$\text{PC}_{\text{new}} = \text{PC}_{\text{branch}} + (\text{Imm} \times 2)$$

For this case:

$$\text{PC}_{\text{new}} = \text{PC}_{\text{branch}} + 32$$

Final Observation:

The waveform confirms correct misprediction handling, where:

- Incorrect speculative instructions are flushed
- ROB state is cleaned properly
- PC is accurately redirected to the branch target

This ensures correct program execution despite speculative execution errors.

6.4. Branch Prediction (BP) Unit Verification

6.4.1. Simulation Console Output

```
# ** Note: TEST 1: Reset behavior
#    Time: 0 ps  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 1
#    Time: 30 ns  Iteration: 0  Instance: /testbench
# ** Note: TEST 2: BTB Miss = Insert Taken
#    Time: 30 ns  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 2 (Insertion done)
#    Time: 40 ns  Iteration: 0  Instance: /testbench
# ** Note: TEST 3: Prediction after insert
#    Time: 40 ns  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 3
#    Time: 50 ns  Iteration: 0  Instance: /testbench
# ** Note: TEST 4: Strengthen to ST
#    Time: 50 ns  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 4
#    Time: 60 ns  Iteration: 0  Instance: /testbench
# ** Note: TEST 5: Not taken transition
#    Time: 60 ns  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 5
#    Time: 70 ns  Iteration: 0  Instance: /testbench
# ** Note: TEST 6: Final Prediction Check
#    Time: 70 ns  Iteration: 0  Instance: /testbench
# ** Note: PASS: TEST 6
#    Time: 80 ns  Iteration: 0  Instance: /testbench
# ** Note: ALL TESTS COMPLETED SUCCESSFULLY
#    Time: 80 ns  Iteration: 0  Instance: /testbench
```

Figure 6.11. ModelSim Console Output Showing BP Unit Verification Tests

The simulation log validates the functionality of the Branch Prediction Unit across multiple test scenarios:

- **Test 1 – Reset Behavior:** BP unit initializes correctly after reset
- **Test 2 – Branch Miss Handling:** Correct insertion of taken branch entry
- **Test 3 – Prediction After Insert:** Accurate prediction using updated state
- **Test 4 – State Strengthening:** Transition towards Strongly Taken (ST) state
- **Test 5 – Not Taken Transition:** Proper state update on not-taken outcome

- **Test 6 – Final Prediction Check:** Final prediction correctness verified

All test cases pass successfully, confirming correct implementation of prediction logic and state transitions.

6.5. Branch Not-Taken Verification

6.5.1. Simulation Waveform

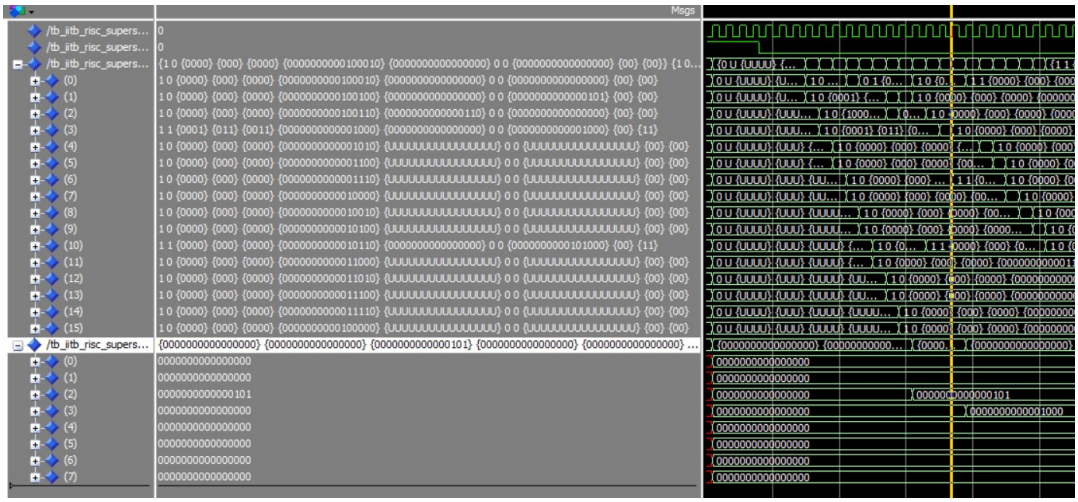


Figure 6.12. Waveform Showing Correct Handling of Not-Taken Branch and Instruction Commit

The waveform verifies the correct behavior of the processor when a branch instruction is predicted as **Not Taken** and the actual outcome also evaluates to **Not Taken**.

In this scenario:

- The branch instruction (**BLE**, Batch 2 – Instruction I1) is predicted as **Not Taken**
- The actual execution confirms the prediction (branch condition evaluates to Not Taken)
- As a result, no pipeline flush occurs and subsequent instructions continue normally
- The following instruction (**ADDI**, Batch 2 – Instruction I2) is allowed to execute and commit

Final Observation:

The successful commit of the ADDI instruction is reflected in the Architectural Register File:

$$R4 = 0000000000001000$$

This confirms that the control flow remains uninterrupted when branch prediction is correct, demonstrating proper pipeline behavior and efficient instruction throughput.

6.6. LMF Instruction Verification

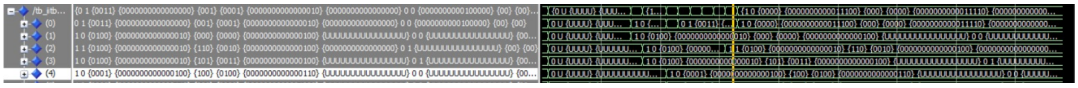


Figure 6.13. Waveform and ROB Table Showing Execution of LMF Instruction

The instruction `x"6306"` corresponds to the **LMF (Load Multiple from Memory)** operation.

From the simulation, it is observed that the instruction sequentially loads values into registers **R2** and **R1**. The data is fetched from consecutive memory locations starting from the base address stored in **R1**.

This behavior is clearly reflected in the ROB table, where both destination registers receive values in order, confirming correct execution of the LMF instruction.

7. Key Design Decisions

7.1. Word-Addressed IMEM with Synchronous Read

The IMEM is word-addressed (16-bit per entry). The PC is used directly as the word index, and two consecutive words are read per cycle. Reads are synchronous (registered), introducing a 1-cycle latency. This models real SRAM/BRAM behaviour and allows higher clock frequency by breaking the long combinational path through the memory array.

7.2. Dual CDB

Two Common Data Buses allow two functional unit results to be broadcast per cycle. Since the processor can issue up to two independent instructions per cycle (one to ALU, one to Branch or LSQ), having dual CDB prevents a broadcast bottleneck.

7.3. Combinational BP Read, Synchronous BP Write

The BTB read is combinational so predictions are available in the same cycle as the fetch address (parallel with IMEM read). Updates are synchronous and happen only on ROB commit — never on speculative execution. This ensures the predictor is never trained on a wrong-path branch.

7.4. TAG Stack for Branch Aliasing

Multiple in-flight branches can alias to the same BTB entry. The TAG Stack tracks in-flight branch PCs. On commit, if the wrong condition is detected (`WC=1`), the stack is cleared and the pipeline flushed, preventing aliasing-induced mispredictions from corrupting the BTB state.

7.5. MF Sequencer for Multi-Register Instructions

Rather than building a complex multi-port datapath for LM/SM/LMF/SMF, the MF Sequencer serialises them into individual micro-operations. The fetch stage is frozen during this sequence (`freeze_fetch=1`), keeping the pipeline clean without structural hazards.

7.6. Zero-Cycle Intra-Bundle RAW Forwarding in RR_Unit

The RR_Unit handles the case where Instruction 2 reads a register being written by Instruction 1 in the same dispatch cycle. The forwarded tag is computed combinationally within the same clock cycle, so no extra stall cycle is needed for same-bundle dependencies.

8. Work Distribution

The project was carried out collaboratively with balanced contributions from all team members. Each member was responsible for major architectural blocks, subsystem integration, testing, debugging, and documentation of the superscalar processor design.

Team Member	**Work Done**
Vinay Hariharan	Designed and implemented the **Fetch Unit, BP Unit, MF Unit, BPT, ALU Reservation Station (ALU RS), and partial Branch Reservation Station**. Contributed significantly to the design of the **ROB (Reorder Buffer)** and branch handling architecture.
Sudhindra Sahoo	Worked on the **Partial Branch Reservation Station, RR Unit, Next Address Block**, and contributed to **ROB** development. Implemented half of the **Decode Unit** and contributed to the **Top-Level Integration** of the processor system.
Himanshu Sahu	Designed and implemented the **LS-RS (Load Store Reservation Station), LSQ (Load Store Queue), LS Adder, Store Buffer**, and system **Flush Logic**. Also developed the **Scheduler** and contributed to half of the **Decode Unit**.
Yashvardhan Khandegar	Designed and implemented the **Dispatch Unit, ALU Execution Unit, Branch Execution Unit, IMEM, DMEM, Register File Array, CDB Arbiter, and CCR Tracker**. Contributed in preparation of the final project report.
All Members	Collaboratively worked on system integration, debugging, simulation, verification of instruction execution scenarios, pipeline hazard handling, and overall processor validation.

Table 8.1. Work Distribution Among Team Members

9. Conclusion

This report has described the complete design of a 2-way fetch, out-of-order execution superscalar processor implementing the IITB-RISC ISA.

- Speculative 2-way instruction fetch with 2-bit saturating counter BTB.
- MF Sequencer for transparent expansion of multi-register memory instructions.
- Full register renaming via RAT and Free List, eliminating WAR/WAW hazards.
- Three parallel execution pipelines: ALU, Branch/Jump, Load/Store.
- In-order commit via ROB with precise misprediction recovery and CCR tracking.
- Dual commit (up to 2 instructions retired per cycle) matching 2-way fetch bandwidth.

The 4-instruction simulation trace confirms correct out-of-order execution: I3 and I4 (dispatched second) complete after I2 due to data dependencies, but the ROB restores in-order architectural state. Dual-commit is demonstrated on cycles 12 and 14 where two instructions retire simultaneously. No mispredictions or pipeline flushes occur in this trace, confirming correct speculative operation for the given instruction sequence.